CREATING WEALTH
FOR WELLBEING

NLC INDIA LIMITED

("NAVRATNA" – A GOVERNMENT OF INDIA ENTERPRISE")

NEYVELI – 607 803, TAMIL NADU

INTERNSHIP REPORT ON

SAP PROJECT SYSTEM USING PYTHON DATASCIENCE TOOLS

SUBMITTED BY

RASIKA.R
SAFIYA BEGAM.K
SHYIVANI.K
SUBATHRA.B
SUGUNA'K

B.TECH INFORMATION TECHNOLOGY

DHANALAKSHMI SRINIVASAN ENGINEERING COLLEGE(AUTONOMOUS)

PERAMBALUR-621212

UNDER THE GUIDANCE OF

VARATHARAJAN T.

M.TECH (CSI), PGDSM, BEC(CAMBRIGE UK), NLC INDIA LIMITED.

CREATING WEALTH
FOR WELLBEING

NLC INDIA LIMITED

(“NAVRATNA” – A GOVERNMENT OF INDIA ENTERPRISE’’)

NEYVELI – 607 803, TAMIL NADU

BONAFIDE CERTIFICATE

Certified that the Internship Training titled “SAP PROJECT SYSTEM REPORTS USING PYTHON DATASCIENCE TOOLS” Enterprise Resource Planning is the Bonafide work of

RASIKA.K
SAFIYA BEGAM.K
SHYIVANI.K
SUBATHRA.B
SUGUNA.K

In INFORMATION TECHNOLOGY DEPARTMENT FROM DHANALAKSHMI SRINIVASAN COLLEGE(AUTONOMOUS), done during the period from 01/07/2025 to 14/07/2025 at Materials Management Complex, NLC INDIA LIMITED.
Their performance, conduct and attendance during the period were found to be ______

Signature Of the guide

Varatharajan T. M.Tech (CSI), PGDSM, BEC(CambridgeUK)

Permitted to submit internship report to college / university authority

DEPUTY GENERAL MANAGER
LEARNING AND DEVELOPMENT CENTRE
NLC INDIA LIMITED,NEYVELI.

DATE:14.07.2025
PLACE: NEYVELI

DECLARATION

I hereby declare that the Internship training titled "SAP PROJECT SYSTEM REPORTS USING PYTHON DATASCIENCE TOOLS" IN NLC INDIA LIMITED NEYVELI, submitted to department of **INFORMATION TECHNOLOGY in DHANALAKSHMI SRINIVASAN ENGINEERING COLLEGE (AUTONOMOUS**) for the degree of Bachelor of Engineering in a reward of original work done by us under the guidance of **Mr. VARATHA RAJAN .T M.Tech (CSI), PGDSM, BEC(Cambridge– UK), DCE/ERP,** NLC India Limited, Neyveli.

This internship report is for reference only and no part of the report will be published copied anywhere without the written permission from officials of NLCIL, Neyveli.

RASIKA.R
SAFIYA BEGAM.K
SHYIVANI.K
SUBATHRA.B
SUGUNA.K

DATE:14/07/2025 **Signature of the student**

PLACE: Neyveli

ACKNOWLEDGEMENT

We express my sincere thank to **NLC INDIA LIMITED** for providing me the opportunity to carry out the Internship Training.

We also express my sincere thank to **Shri.A.Kr.SARAVANABHAVAN, GM/L&DC & Shri.K.PRABAKARAN/DGM/L&DC for** learning & development center providing me the opportunity to carry out the

Internship Training in NLCIL,NEYVELI.

We sincerely thank to **Shri.VARATHARAJAN.T M.Tech (CSI), PGDSM, BEC(Cambridger – UK) (DCE/ERP)** for providing the guidance for the internship training programme of **2 weeks** during the period of **(From: 01/07/2025 To: 14/07/2025)** in NLCIL, NEYVELI. Special Thanks to **Shri.SOLOMAN LUTHER KING (GM(CS&ERP)** and **Shri.A.MANIMEKALAI (GM/ERP)**.

We sincerely thank the officials in the Field Administrative office for providing permission to enter the Material Management Complex.

I sincerely thank all the respectful diginitaries for providing great support for successfully completing the internship training in the NLCIL,NEYVELI.

INDEX

CHAPTER-I

INTRODUCTION

The main activity of NLC India Limited is Mining (Coal & Lignite) and Power Generation (Thermal and Renewable Energy).

Mining

NLC India at present has four open cast lignite mines namely Mine I, Mine II, Mine IA & Barsingsar Mine and one open cast coal mine, Talabira II & III. The lignite mined out is used as fuel to the linked Pit head power stations. Also raw lignite is being sold to small scale industries to use it as fuel in their production activities.

Lignite Mining	
Mine I, Neyveli	8 MTA
Mine I A, Neyveli	7 MTA
Mine II, Neyveli	15.0 MTA
Barsingsar Mine	2.1 MTA

Coal Mining	
Talabira II & III OCP	20.0 MTA

Power Generation

NLC India has five pithead Thermal Power Stations with an aggregate capacity of 3640 MW. Further, NLC India has installed 51 MW Wind Power plant, commissioned 1350.06 MW Solar Photo Voltaic Power plant in Tamil Nadu and also commissioned 20 MW Solar Plant in Andaman & Nicobar islands, resulting in an overall power generating capacity of 5061.06 MW (excl. JVs).

Thermal Power	
TPS - II, Neyveli	1470 MW
TPS - I Expansion, Neyveli	420 MW
NNTPS, Neyveli	1000 MW
TPS, Barsingsar	250 MW

	Wind Power	
	Wind Power Plant	51 MW

	Solar Power	
	Solar Power Plant	1370.06 MW

NLC Tamil Nadu Power Limited (NTPL) (2x500 MW)

NLC Tamil Nadu Power Limited (NTPL), is a joint venture company of NLC India Ltd (formerly known as NLC Ltd) and M/s TANGEDCO (Tamil Nadu Generation and Distribution Company), incorporated under the company act. The Equity participation between NLCIL and TANGEDCO is at the ratio of 89:11. GOI had issued sanction for the implementation of coal based 2 X 500 MW Thermal Power Project by NTPL at Tuticorin. Unit 1 and Unit 2 have been declared for commercial operation w.e.f. 18th June 2015 and 29th August 2015. Power Purchase Agreement has been signed with TANGEDCO, ESCOMs of Karnataka State, Puducherry Electricity Department, Kerala State Electricity Board and DISCOMs of Andhra Pradesh. Power evacuation from this project is being carried out by M/s Power Grid Corporation of India. Upon commencement of coal production from linked Talabira Mines, the Mine started to supply coal to its end use Plant, NTPL TPS, Thoothukudi

Neyveli Uttar Pradesh Power Limited (3x660MW)

Neyveli Uttar Pradesh Power Limited (NUPPL) is a joint venture between NLC India and Uttar Pradesh Rajya Vidyut Utpadan Nigam Limited, Ghatampur in the State of Uttar Pradesh, for setting up of 3 x 660 MW power project. The Equity participation between NLCIL and UPRVUNL is at the ratio of 51:49. MoC accorded approval for the transfer of 20% equity out of 49% equity share held by UPRVUNL in NUPPL in favour of Govt. of Assam on 31.05.2023. LOA for Steam Generator (SG) and Turbo Generator (TG) and Balance of Plant packages have been issued. Erection works under progress. Mock light up of Unit-I was successfully demonstrated on 26th March, 2021. For Unit 1, Boiler Light up was completed on 28.02.2022 and Steam Blowing Light Up on 13.08.2022. Expected Commissioning of Unit-I by 31st July 2023.

Coal Lignite Urja Vikas Private Limited (CLUVPL) – A Joint Venture Company between NLCIL &CIL

JV Company between NLCIL & CIL "Coal Lignite Urja Vikas Pvt Ltd" was incorporated on 10.11.2020 with equity participation of 50:50.

The Company had received an in-Principle approval from South Eastern Coalfields Limited (SECL) to engage CLUVPL as PMC service provider for installation of 40 MW Solar Project in their lands at Bhatgaon and Bishrampur area, Chhattisgarh. The project is under execution.

NLC India Renewables Ltd. – Fully owned JV

Wholly Owned Subsidiary Company, "NLC India Renewables Limited" has been incorporated on 14.06.2023 to take over existing renewable assets of NLCIL. One more JV is planned under this JV for taking-up all the future renewables projects.

Bithnok Thermal Power Project (250 MW) with linked Bithnok Lignite Mine (2.25 MTPA), Rajasthan

Bithnok Thermal Power Project (250 MW) with the linked lignite mine of 2.25 MTPA capacity at Bithnok in the State of Rajasthan was planned to setup at an aggregate estimated cost of Rs.2709.93 Cr. The project has been put on hold since June 2017. However, as part of the revival of the same, based on the discussion with Rajasthan Government officials, it has been agreed that NLCIL and GoR will jointly work to revive the Bithnok Thermal Power Project along with Bithnok Mines. Feasibility study report prepared and the same is under scrutiny & approval.

Pachwara South Coal Blocks (09 MTPA)

NUPPL has been allotted with the Pachwara South Coal Block, in the State of Jharkhand, with a capacity of 9.0 MTPA (Normative) &13.50 MTPA (Peak) at an estimated cost of Rs. 1795 Cr. In order to develop and operate the above Coal Block, MIPL GCL Infra contract Private Limited has been appointed as the Mine Developer Operator (MDO). Geological Report (GR), Mining Plan & Mine closure plan approved by MoC.Project under development stage. End user: Ghatampur Thermal Power Project (3x660 MW), UP

NLC Talabira TPP (NTTPP) Phase-I (3 X 800 MW)

NLCIL Board accorded in principle approval for taking up 2400 MW Coal based pit head Power Station at Tareikela, Jharsuguda District in Odisha. Administrative approval for acquisition of private land was received from Government of Odisha. Environmental Clearance obtained on 02.02.2021. Single EPC tender opened and Evaluation under progress. Land acquisition is under progress

Thermal Power Station -II Second Expansion 2x660 MW with linked Mine-III (11.5 MTPA), Neyveli Tamil Nadu.

It is proposed to increase the power generating capacity by adding another 1320 MW thermal power plant as the second expansion to the existing TPS-II at Neyveli. A new mine, Mine-III of peak capacity of 11.5 MTPA is proposed to be set up to exploit the mineable lignite reserves of about 415 MT available in the south of the existing Mine-II to meet the fuel requirement of the proposed thermal power plant. Tender floated for award the work on EPC basis.

510 MW solar power project under CPSU scheme

NLCIL participated in IREDA CPSU scheme tender and emerged as a successful bidder for setting-up 510 MW solar power project (10 MW Solar Project for Mini Smart City) with Quoted Viability Gap Funding (VGF) of Rs 44,74,990 /- per MW. LoA was received on 04.10.2021.

10 MW Solar Project (Mini Smart City):

The Project is planned to meet the minimum requirement norm of MNRE to make Neyveli Township as Mini Smart City. Project was synchronised with the Grid on 23.06.2023. Project is likely to be completed by July 2023.

300 MW Solar Project:

LoA issued on 27.03.2023 to M/s Tata at a project cost of Rs. 1755 Cr. at Barsingsar in the land available with NLCIL. Works has been started.

Consent from RUVNL is obtained on 23.08.2022 for usage of 300 MW CPSU solar power from NLCIL. PPA signing is under progress.

200 MW Solar Project:

Tender floated for Pan-India installation.

150 MW WIND POWER PROJECT PAN INDIA (SECI)

NLCIL participated and bagged SECI Hybrid tender for 150 MW (Wind + Solar). Tender opened for both Solar and Wind and it is under evaluation.

New Solar Projects

Ground Mounted Solar Plant in Mine Re-claimed area:

Tender floated for the installation of 50 MW Solar power projects in Reclaimed area of Mines. Further, it is planned to install another 300 MW in vacant land.
Also, NLCIL is planning to install 100 MW floating solar & 6 MW roof top solar projects. Activities are under process.

LIGNITE TO METHANOL

To utilise the large reserves of lignite in environment friendly manner, NLCIL proposed to install Lignite to Methanol project of 1200 TPD (0.4 MTPA) capacity by utilising 2.26 MTPA.
Two tenders were floated – one for gasification block and another for Methanol block. Bid opening is scheduled for 7th August for both bids.

OVERBURDEN TO SAND

NLCIL Proposes to extract construction grade sand from the its all three Mines overburden dump.
LOA issued for setting up of pilot plant in Mine-IA [2.62 Lcum (0.42 MTPA)],
Tender floated for setting up of pilot plant in Mine-I [6.25 Lcum (1.0 MTPA)].

On Pilot Scale Projects Planned

Battery Storage System: A&N administration requested to check the feasibility for installing another 20 MWhr Battery Energy Storage System (BESS) in addition to the existing 8 MWHr BESS. SECI will float the tender, NLCIL will participate.

Pumped Storage System: MoU signed with M/s WAPCOS on 27.05.2023, to develop Hydro-Power Projects, Pumped Storage and Reservoirs.

Lignite to Syngas: MoU signed with BHEL on 12.10.2022 for taking up Joint collaborative studies with M/s. BHEL to study the feasibility of setting up a pilot plant.Further activities are under progress.

Lignite to Diesel: M/s LEMAR, USA prepared Feasibility Report (FR) on conversion of lignite to diesel and declared that Lignite is good for the gasification and processing into diesel fuel. Further activities are under progress

Green Hydrogen: NLCIL p3337lanned for setting-up of a Pilot scale Project to generate Hydrogen with in-put of 4 MW Solar to the Electroliser

About Enterprise Resource Planning (ERP)?

Enterprise resource planning (ERP) is a suite of integrated applications that helps organizations manage their core business processes, such as accounting, manufacturing, sales, and human resources. ERP tools provide a single source of truth for all of an organization's data, which can help to improve efficiency, accuracy, and visibility.

ERP Tools in the Market

There are many different ERP tools available on the market, each with its own set of features. Some of the most popular ERP tools include:

- **Oracle ERP:** Oracle ERP is one of the most comprehensive ERP solutions on the market. It offers a wide range of features, including financial management, supply chain management, manufacturing, and human resources.
- **SAP ERP:** SAP ERP is another popular ERP solution. It is known for its scalability and flexibility. SAP ERP can be customized to meet the specific needs of any organization.
- **Microsoft Dynamics 365:** Microsoft Dynamics 365 is a cloud-based ERP solution that offers a wide range of features, including financial management, sales, and customer relationship management (CRM).
- **NetSuite ERP:** NetSuite ERP is a cloud-based ERP solution that is known for its ease of use and scalability. NetSuite ERP is a good choice for businesses that are looking for a flexible and affordable ERP solution.

Acumatica ERP: Acumatica ERP is a cloud-based ERP solution that is known for its customization capabilities. Acumatica ERP is a good choice

.for businesses that need a flexible and scalable ERP solution that can be customized to meet their specific needs.

Features of ERP Tools

The features of ERP tools vary depending on the specific solution. However, some of the most common features include:

- Financial management: ERP tools typically include modules for accounting, budgeting, and forecasting.

- Manufacturing: ERP tools can help to manage the manufacturing process, including planning, scheduling, and inventory control.
- Sales and marketing: ERP tools can help to manage the sales and marketing process, including lead generation, customer relationship management (CRM), and order management.
- Human resources: ERP tools can help to manage the human resources process, including payroll, benefits, and recruitment.

Benefits of ERP Tools

ERP tools can provide a number of benefits to businesses, including:

- Improved efficiency: ERP tools can help to improve efficiency by automating many of the manual tasks that are involved in running a business.
- Increased accuracy: ERP tools can help to improve accuracy by providing a single source of truth for all of an organization's data.
- Enhanced visibility: ERP tools can help to improve visibility by providing real-time data on all of an organization's operations.
- Better decision-making: ERP tools can help to improve decision-making by providing insights into an organization's performance.

Choosing an ERP Tool

If you are considering implementing an ERP tool, it is important to select a solution that meets the specific needs of your organization. You should also consider the cost of the solution, the level of support that is available, and the ease of implementation.

About SAP.

SAP stands for Systems, Applications, and Products in Data Processing. It is a German multinational software corporation that develops enterprise software. SAP's best-known product is SAP ERP, which is an enterprise resource planning (ERP) system.

SAP ECC 6.0

SAP ECC 6.0 is a version of SAP ERP that was released in 2005. It is a comprehensive ERP system that offers a wide range of features, including financial management, manufacturing, sales, and human resources. SAP ECC 6.0 is a modular system, which means that it can be customized to meet the specific needs of any organization.

Functional SAP ERP Modules:

- Project System (SAP PS)
- Production Planning (SAP PP)
- Quality Management (SAP QM)
- Sales and Distribution (SAP SD)
- Material Management (SAP MM)
- Financial Supply Chain Management (SAP FSCM)
- Financial Accounting and Controlling (SAP FICO)
- Human Resource Management (SAP HRM)
- Human Capital Management (SAP HCM)
- Logistics Execution (SAP LE)
- Business Information Warehousing (SAP BIW)
- Customer Relationship Management (SAP CRM)
- Supplier Relationship Management (SRM)

Technical SAP ERP Modules:

- SAP Basis
- SAP Security

- SAP NetWeaver
- SAP Solution Manager
- Exchange Infrastructure (SAP XI)
- High Performance Analytic Appliance (SAP HANA)
- Information Systems Management (SAP IS)
- Customer Relationship Management (SAP CRM Technical module)
- Advanced Business Application Programming (SAP ABAP)

Let's take a closer look at each of Functional SAP ERP modules and their features in more detail.

1. SAP PS Project System

The SAP PS module enables you to quickly create, manage, and track project assignments. It enables users to collaborate and share resources across teams, manage schedules and all aspects of a project from start to finish.

In project management module that allows you to plan, assign, manage, and monitor your projects. You can quickly access project information from anywhere in the organization. The system's electronic docs and workflows make it easy to generate reports and manage project updates. You can also use the SAP PS to manage and control project costs.

PS offers the following features:

Projects: SAP PS enables you to quickly create, manage, and track project assignments. You can quickly access project information from anywhere in the organization.

Project management: This module allows you to plan, assign, manage, monitor, and conform your projects.

Electronic docs and workflows: SAP PS makes it easy to generate reports and manage project updates.

Project costs: SAP PS enables you to control and manage project costs.

Organizational cubes: SAP PS allows you to view and manage project information within the organization's cubes.

Mailchimp integration: This system offers integration with Mailchimp, an email marketing platform.

2. Production Planning (SAP PP)

Production Planning (SAP PP) is a key SAP ERP Module that helps manage the production process. It involves activities such as planning and forecasting, managing resources, and setting up workflows.

Major activities involved in SAP PP:

Planning and forecasting: The first step in production planning is to plan the production process. This involves determining the sequence in which the resources will be used, as well as the resources that will be needed to produce a particular product.

Allocating/Managing resources: Once the production process has been planned, the next step is to allocate the resources needed to produce the product. This involves deciding which resources will be used in which sequence, as well as determining the number of resources that will be needed to produce the product.

Monitoring and controlling the production process: Once the resources have been allocated and the production process has been started, it is important to monitor and control the process. This involves making sure that the products are produced in a timely and efficient manner, as well as ensuring that the products meet customer expectations.

Adjusting the production process as needed: If the production process is not working as planned, it is necessary to adjust it. This can be done by changing the sequence of resources or the number of resources that are used.

3. SAP Quality Management (SAP QM)

Quality management in SAP systems is essential to ensure that the quality of services and products delivered to customers is consistently high. It covers the entire process from Quality Control, Planning, Assurance, Reporting and identification of quality risks to corrective and preventive actions.

The following are some of the key quality management features in SAP ERP:

Defining quality objectives and constraints: Quality objectives and constraints are established to ensure that the delivered services are meeting customer requirements and comply with applicable regulatory requirements.

Controlled and monitored process flow: Processes and systems are controlled and monitored to ensure that they produce the required quality products and services. This includes quality checks and audits at various points in the process.

Identification of quality risks: Quality risks are identified based on the potential consequences of their occurrence. This allows preventative actions to be taken to mitigate their effects.

Corrective and preventive actions: Corrective and preventive actions are taken to address quality risks and improve the quality of products and services delivered to customers.

4. SAP Material Management (SAP MM)

SAP MM is a powerful SAP ECC 6.0 tool to manage resources, tracking inventory and materials. It is an extremely important aspect of any enterprise, and is essential for ensuring that materials are available when they are needed and that costs are kept to a minimum.

In connection with the various lifecycle phases of the business, accumulating and managing material is of utmost importance. The SAP ECC 6.0 material management solution delivers a comprehensive view of all material in the system and enables organizations to efficiently navigate through material inventory and track material cycles.The SAP ECC 6.0 material management solution can help organizations achieve the following objectives:

- Limit waste and optimize material usage through comprehensive view of inventory and cycles
- Streamline Procurement and Delivery processes with visibility on material status.
- Optimize material extraction, reduction, and recycling efforts.

Features of the SAP ECC 6.0 material management solution include the following:

- Material Inventory

- Asset Management
- Material Portfolio Management
- Material Cycle Management
- Material Status
- Resource Planning
- Material Extraction, Reduction, and Recycling
- Material Usage Analysis
- Real-Time Decision Making

5. SAP Financial Supply Chain Management (SAP FSCM)

SAP FSCM is a key enabler of SAP ECC 6.0, SAP FSCM helps Organizations manage key aspects of their supply chains, including procurement, inventory management, and shipping, ensuring that products and services are delivered on time and in budget. With its broad range of capabilities.

Planning tools help you create a roadmap and define your business goals. Execution tools help you track and manage your supply chain activities.

Successful FSCM begins with an accurate and timely forecast of future needs and sufficient resources to meet them.

SAP FSCM Module have a number of features that help you achieve your goals.**Monitor and manage spend:** Know precisely where your money is going and make smart choices based on budget implications.**Track customer spending:**Monitor your customers' spending habits and see where they are spending the most.

Track financial options: Get deep insights into your company's funding options and make the best financial decisions for your business.

Optimize logistics: Get a full picture of your supply chain and optimize your inventory to minimize costs.

Improve cash flow: Create detailed and accurate forecasting of future requirements, and take necessary steps to get funding when you need it.

Stay compliant: Stay compliant with regulatory requirements and stay in front of changing trends.

6. SAP Financial Accounting and Controlling (SAP FICO)

SAP ECC FICO module is another important SAP ERP ECC 6.0 modules offer to enterprises, that is highly scalable, making it easy to manage your organization's financial resources.The benefits of using SAP ECC FICO include:

- Improved efficiency and accuracy of financial management
- Improved decision-making in financial planning and forecasting
- Increased compliance with international accounting standards
- Improved transparency and understanding of financial statements

Here is the list of some components and their features and benefits:

General ledger: In general ledger accounting, you track the financial transactions of your business. This includes recording the liabilities and assets of your company, as well as the changes in their respective balances. SAP FICO helps you to keep track of your company's finances in a simple and organized manner, making it easier to make informed decisions.**Accounts receivables:** Accounts receivables are the money your business has borrowed from suppliers or customers. In order to ensure that you comply with your financial obligations, you need to track the progress of your receivables and ensure that you have sufficient funds to meet your debts. SAP FICO can help you to do this by tracking your receivables and identifying any irregularities or discrepancies.**Accountspayables:**Accountspayablesarethemoneyyourbusinessowes to other companies or individuals. They are usually represented by a debit balance in your company's account books. Once you have received payments from your customers, you need to track this in order to ensure that you have the correct accounts open and that you are not overspending on supplies.

Asset accounting: Asset accounting is a key function in accounting for your business's physical assets. This includes everything from computers and furniture to company vehicles and intellectual property. SAP FICO can help you to track the value and location of your assets, as well as to value and assess any improvements or changes to your assets.

Bank accounting: Bank accounting is used to track the financial transactions of your business with your bank. This includes recording the source of your funds and the transactions that take place between your company and the bank.

It is important to keep track of your company's bank accounts in order to ensure that you are complying with all your financial obligations.

Comprehensive Automatic Fraud Detection: SAP ECC 6.0 includes powerful automatic fraud detection capabilities to help you identify and prevent financial fraud. This helps you maintain integrity and guard against financial losses.

7. Human Resource Management (SAP HRM)

SAP ECC Human Resource Management module offers a comprehensive platform for managing people in organizations of all sizes.SAP HRM module provides complete human resources management capabilities, including recruitment and staffing, workforce planning, organizational management, benefits administration, payroll employee relations, and performance management which will help your organization streamline operations and improve employee satisfaction.Here are some of the key features of SAP ECC Human Resource Management module:

Planning

SAP ERP ECC 6.0 HRM provides the ability to plan for and forecast future needs for employees, including hiring, training, and staffing. This can help you identify and address potential problems early on, saving your organization time and money in the long run.

Recruitment

SAP ECC 6.0 HRM allows you to manage your recruitment process from start to finish, from finding and vetting candidates to placing them into the right positions. This can help you keep your recruitment costs low and ensures that you’re getting the best candidates for your organization.

Staffing

SAP HRM allows you to easily identify and fill open positions, whether you’re looking for new employees or filling personnel cuts. This can help you keep your organization running smoothly and avoid long-term staffing shortages.

Employee Relations

SAP ECC HRM helps you to manage your workforce effectively, from managing employee complaints and confrontations to setting employee performance goals.

This can help you build a productive and positive environment for your employees.

Performance Management

SAP HRM offers features like performance appraisals and rewards programs to help you manage employee performance. This can help you provide employees with accurate and timely feedback, helping them improve their skills and productivity.

8. CRM Customer Relationship Management (SAP CRM)

SAP ECC CRM (Customer Relationship Management) is a module of SAP that helps businesses manage customer interactions and data. It enables organizations to track and respond to customer needs and preferences in a timely manner.

The main features of SAP CRM include the following:

Lead Management:

The lead management functionality of SAP CRM allows businesses to track and manage leads throughout the sales cycle. It helps sales teams identify and prioritize leads, and track their activities and progress.

Contact Management:

The contact management functionality of SAP CRM enables businesses to manage their customer contact information in a central repository. It helps organizations keep track of customer interactions, and respond to customer inquiries in a timely manner.

Customer Data Management:

SAP CRM provides a central repository for all customer data. This includes contact information, demographic data, and purchase history. SAP CRM also provides tools for managing customer data quality.

Opportunity Management:

The opportunity management functionality of SAP CRM allows businesses to track and manage sales opportunities throughout the sales cycle. It helps sales teams identify and prioritize opportunities, and track their progress.

Case Management:

The case management functionality of SAP CRM enables businesses to track and manage customer service cases. It helps customer service teams resolve customer issues in a timely and efficient manner.

Product Catalogue:

The product catalogue functionality of SAP CRM enables businesses to manage their product information in a central repository. It helps organizations keep track of product details, and price changes.

Price List:

The price list functionality of SAP CRM enables businesses to manage their pricing information in a central repository. It helps organizations keep track of pricing changes, and discounts.

Workflow:

The workflow functionality of SAP CRM enables businesses to automate their business processes. It helps organizations to streamline their business operations, and improve efficiency.**Automated Customer Service and Support:**SAP ERP ECC CRM includes a number of features for automating customer service and support processes. This includes a knowledge base, case management, and web self-service.

Analytics and Reporting:

SAP ECC CRM provides a number of tools for analyzing customer data and generating reports. This includes predictive analytics, customer segmentation, and reporting.

9. Human Capital Management (SAP HCM)

SAP ERP HCM is a power tool which helps businesses manage their human capital. It offers a suite of tools for tracking employee information, managing payroll and benefits, and managing talent and performance.

SAP HCM is a powerful tool that can help companies streamline their HR processes and improve their overall efficiency. Here are some of the key features of SAP HCM:

Payroll and benefits administration: SAP HCM can help companies automate their payroll and benefits processes. This can save a lot of time and money, and help ensure that employees are paid correctly and on time.

Performance management: SAP HCM includes a powerful performance management system. This can help companies track employee performance and identify areas where improvement is needed.

Recruiting: SAP HCM can help companies streamline their recruiting process. It includes features such as job posting and applicant tracking.

Employee self-service: SAP HCM includes a self-service portal that allows employees to view and update their personal information, access their payroll information, and more.

SAP ECC 6.0 HCM is definitely worth considering if you are searching for a comprehensive human capital management solution.

10. Logistics Execution (SAP LE)

SAP Logistics Execution (SAP LE) module is a part of the SAP ECC ERP system that covers all aspects of logistics execution. It provides a complete picture of the supply chain, from goods receipt to shipment, and enables businesses to plan and execute their logistics activities in a more efficient and effective way including

- Centralized management of logistics execution
- Real-time visibility into all aspects of logistics operations
- Improved coordination between different parts of the supply chain
- Automated execution of logistics processes
- Reduced costs and improved efficiency

SAP LE is an integrated module that helps businesses to effectively manage their logistics processes. The module provides a central platform for managing all activities involved in logistics execution, from transportation and warehousing to order management and track & trace.

Some of the key features of SAP LE include:

Transportation management: SAP LE helps businesses to plan and execute their transportation activities in a more efficient way. It provides a complete view of the transportation network and helps businesses to optimize their routes and resources.

Warehouse management: SAP LE provides a complete picture of the warehouse, from incoming goods to outgoing shipments. It helps businesses to optimize their warehouse operations and to reduce their inventory levels.

Order management: SAP LE helps businesses to manage their orders in a more efficient way. It provides a complete view of the order process, from creation to delivery, and helps businesses to track and trace their orders.

Track and trace: SAP LE provides a complete picture of the supply chain, from goods receipt to shipment. It helps businesses to track and trace their shipments and to monitor the status of their orders.

ABAP – Programming Language. ABAP (Advanced Business Application Programming) is a high-level programming language created by SAP SE for the development of business applications in the SAP environment. It is a fourth-g
eneration language (4GL)
that is event-driven and procedural. ABAP is also object-oriented, meaning that programmers can use objects to represent real-world entities and relationships.
ABAP is used to develop a wide variety of business applications, including:

- Reports
- Interfaces
- Forms
- Data conversions
- User Exits & BADIs
- Workflows
- Business intelligence applications

ABAP is a powerful language that can be used to create complex and sophisticated business applications. It is also a relatively easy language to learn, making it a good choice for both experienced programmers and those new to programming.

Here are some of the key features of ABAP:

- **Event-driven:** ABAP programs are driven by events, such as user actions or system events. This makes ABAP well-suited for developing interactive applications.

- **Procedural:** ABAP programs are written in a procedural style, meaning that they are a sequence of instructions that are executed one after the other. This makes ABAP easy to understand and debug.
- **Object-oriented:** ABAP is an object-oriented language, meaning that programmers can use objects to represent real-world entities and relationships. This makes ABAP well-suited for developing complex applications.

Integrated database access: ABAP has integrated database access, meaning that programmers can access data directly from the database

- without having to write complex SQL queries. This makes ABAP a powerful tool for developing data-driven applications.

ABAP is a widely used language and there is a large community of ABAP programmers around the world. There are also many resources available to help programmers learn ABAP, such as books, online courses, and tutorials.

If you are interested in developing business applications for SAP, then ABAP is a good language to learn. It is a powerful and versatile language that can be used to create a wide variety of applications.

Python – Programming Language.

Python is a high-level, general-purpose programming language. Its design philosophy emphasizes code readability with the use of significant indentation. Python is dynamically typed and garbage-collected. It supports multiple programming paradigms, including structured (particularly procedural), object-oriented and functional programming.

Python is a popular language for a wide variety of tasks, including:

- Web development
- Data science
- Machine learning
- Artificial intelligence

- Scientific computing
- Game development
- System administration
- Scripting

Here are some of the key features of Python:

Easy to learn: Python has a simple syntax that is similar to the English language. This makes it easy for beginners to learn and use

- **Powerful:** Python is a powerful language that can be used to create complex applications.
- **Flexible:** Python supports multiple programming paradigms, making it a versatile language that can be used for a wide variety of tasks.
- **Portable:** Python code can be run on a variety of platforms, making it a good choice for cross-platform development.
- **Free and open-source:** Python is a free and open-source language, which means that it is available to everyone to use and modify.

Python is a popular language with a large community of users and developers. There are many resources available to help people learn Python, including books, online courses, and tutorials.

If you are looking for a powerful, flexible, and easy-to-learn programming language, then Python is a good choice.

Here are some of the benefits of using Python:

- Fast development: Python is a fast language to develop in, which means that you can get your projects up and running quickly.
- Efficient: Python code is often very efficient, which means that your applications will run quickly.
- Robust: Python is a robust language, which means that it is less likely to crash or produce errors.
- Portable: Python code can be run on a variety of platforms, which makes it a good choice for cross-platform development.

- Free and open-source: Python is a free and open-source language, which means that it is available to everyone to use and modify.

If you are looking for a language that can help you to be productive, efficient, and reliable, then Python is a good choice.

Our Choice of Programming Language

SAP has traditionally used the ABAP programming language for various applications. However, in recent years, there has been a growing interest in using Python for SAP development. This is due in part to the increasing number of Python modules that support artificial intelligence and machine learning, as well as the fact that SAP themselves now recommend Python for certain applications.

One of the key benefits of using Python for SAP development is the DRY (Do not Repeat Yourself) concept. DRY is a software development principle that encourages programmers to avoid duplicating code. This can help to improve the readability, maintainability, and scalability of code.

Another benefit of using Python for SAP development is the SAP Office Python interface. This interface allows Python programs to call SAP BAPIs (Business Application Programming Interfaces). BAPIs are a set of standard interfaces that allow programmers to access SAP data and functionality.

In conclusion, there are a number of benefits to using Python for SAP development. These benefits include the support for artificial intelligence and machine learning, the DRY concept, and the SAP Office Python interface. As a result, Python is becoming an increasingly popular choice for SAP development.

Specifically, in our project, I used Python to call all the BAPIs of SAP. This allowed me to solve a number of problems that would have been difficult or impossible to solve with ABAP.

I believe that Python is a powerful tool that can be used to improve the efficiency and effectiveness of SAP development. I encourage other developers to consider using Python for their SAP projects.

The following external modules of Python was used for the problem coding.

1. OS
2. Openpyxl
3. Pandas

4. Numpy
5. Dateutils
6. Plotly
7. Dash
8. Flask
9. PyQt
10. Selenium, Etc.

Objectives of the Training

Introduction to Computer Science and Programming in Python:

This course introduces programming fundamentals using Python, emphasizing hands-on learning, problem-solving, and computational thinking. Students are encouraged to actively practice by following lecture slides and experimenting with code

Computation & Machine Architecture:

Computation involves following step-by-step instructions (imperative knowledge). Computers are stored-program machines with memory, I/O, ALU (operations), and a control unit (controls flow). Primitive operations include arithmetic, logic, and data movement.

Python Objects & Types:

Python treats everything as an object. Scalar types include int, float, bool, and None. Strings are immutable sequences of characters. Tuples are immutable, ordered sequences; lists are mutable with many methods like append(), sort(), and slicing. Dictionaries store key-value pairs and are mutable, efficient for lookups and frequency counting.

I/O, Variables & Expressions:

Use print() for output, input() for user input. Variables are assigned using

=.Python supports arithmetic, comparison, and logical operations. Expressions follow operator precedence.

Control Flow & Loops:

Conditionals (if/elif/else) control decision-making. while loops repeat as long as conditions are true; for loops iterate over ranges or sequences. Use break to exit loops early.

Functions & Scope:

Functions support modularity and abstraction. Defined with def, they take parameters and may return values. Each call creates a new scope. Functions can have default arguments and be passed or returned as values.

Object-Oriented Programming:

Classes define new data types. Objects (instances) combine attributes and methods. The __init__ method initializes data, and __str__ defines string output. Inheritance allows one class to extend another, and operator overloading enables custom behavior for operators like + or ==.

Algorithm Analysis:

Efficiency is analyzed using Big O notation: O(1), O(log n), O(n), O(n log n), O(n^2), etc. Binary search is O(log n), merge sort is O(n log n), and bubble/selection sort are O(n^2). Recursive algorithms (e.g., Fibonacci) can be exponential if not optimized.

Testing & Debugging:

Testing ensures correctness. Unit tests test individual functions, integration tests test combined modules, and regression tests check for new bugs. Use assert statements for internal checks and exceptions (try/except) for handling errors. Avoid silently ignoring errors.

Debugging strategies include systematic isolation, using print(), bisection,, and visual tools. Syntax, semantic, and logic errors must be distinguished. Python raises exceptions on invalid operations, which should be handled properly.

Assertions & Error Handling:

Assertions validate assumptions (e.g., assert x > 0). Use try/except blocks for expected errors and finally for cleanup. Raising custom exceptions signals specific error conditions. Avoid using return values as error indicators.

Course Goal:

The course develops students' computational thinking, problem-solving, and coding skills by focusing on abstraction, algorithms, and automating tasks using precise language and logic.

Web Programming with Python and JavaScript :

This course explores web application development using Python and JavaScript, focusing on both frontend and backend technologies. It integrates theory with hands-on projects, culminating in a custom final project.

Frontend Development:

HTML5 structures web pages using tags like headings, lists, forms, and tables. The DOM represents page structure hierarchically. CSS3 styles pages with properties for layout, color, fonts, padding, margin, and supports responsive design via media queries, Flexbox, and Grid. Sass adds variables and nesting for maintainability.

Backend Development with Python & Django:

Python introduces programming fundamentals, including control structures, data types (lists, dictionaries, sets), functions, OOP, exception handling, and modules. Django, a high-level Python framework, supports rapid development with MVC-like architecture. It uses views to handle requests, templates for dynamic HTML rendering, URL routing, forms for user input, sessions for user data, and models with an ORM for database access. Django provides built-in admin tools and authentication systems.

Version Control with Git:

Git tracks code changes with commands like `clone`, `add`, `commit`, `push`, and `pull`. GitHub supports collaboration via issues andpullrequestsBranching facilitates parallel development and conflict resolution.

Databases & SQL:

SQL manages relational data via tables. Commands include `CREATE`, `INSERT`, `SELECT`, `UPDATE`, and `DELETE`. It uses filters (`WHERE`, `AND`, `LIKE`), joins, and constraints (`PRIMARY KEY`, `FOREIGN KEY`, `NOT NULL`). Django models abstract SQL operations, enabling data relationships and migrations.

Client-Side Development with JavaScript & React:

JavaScript manipulates the DOM, handles events (`onclick`, `onkeyup`), fetches data from APIs using `fetch()` and JSON, and stores data in `localStorage`. React builds reusable components with JSX, manages state via `useState`, and creates dynamic SPAs with declarative UI updates and routing.

Software Engineering Practices:

Testing (e.g., `unittest`, Selenium) ensures code reliability. CI/CD automates integration and deployment, using GitHub Actions. Docker containers standardize development environments with `Dockerfile` and `docker-compose`.

Scalability:

Web applications scale via vertical (stronger servers) or horizontal (more servers) scaling, supported by load balancers, sticky sessions, and auto-scaling. Databases scale via sharding and replication. Caching (client-side or server-side) reduces load and improves speed.

Security:

Secure development includes avoiding credential leakage (use environment variables), password hashing, HTTPS encryption, and defenses against SQL injection, XSS, CSRF, and spoofing. CSRF tokens and rate limiting enhance protection. Authentication and permissions enforce access control.

Conclusion:

The course integrates multiple technologies and best practices to build scalable, maintainable, and secure web applications, leveraging Python and Java Script for full-stack development.

CONTENT OF STUDY

INTRODUCTION TO COMPUTER SCIENCE AND PROGRAMMING IN PYTHON

Computation:

Introduction to basic computational concepts and how computers solve problems.

Branching and Iteration

Concepts of conditional statements (if, else) and loops (for, while) in Python.

EXAMPLE:

```
  while True: x = float(input("Enter a number for x: "))
  y = float(input("Enter a number for y: "))
  if x == y:
    print("x and y are equal")
  if y != 0:
    print("Therefore, x / y is", x / y)

elif x < y:
   print("x is smaller")
```

```
elif x > y:
   print("y is smaller")
print("Thanks!")
# Ask if user wants to continue
choice = input("Do you want to try again? (yes/no): strip().lower()
if choice != 'yes':
    print("Exiting the program.")
    break
```

String Manipulation, Guess and Check, Approximations, Bisection

Working with strings and solving problems using methods like guess-and-check, approximation, and bisection search.

Decomposition, Abstractions, Functions

Breaking problems into smaller parts, creating abstractions, and defining reusable functions.

Tuples

A tuple is an ordered, immutable collection of items. Once created, you cannot change its contents.

EXAMPLE

```
   # Empty tuple
t0 = ()
# Single-element tuple (note the trailing comma)
t1 = (42,)
# Multiple elements
t2 = (1, 2, 3)
# Tuple without parentheses (tuple packing)
t3 = 4, 5, 6
print(t0, t1, t2, t3)
# Output: () (42,) (1, 2, 3) (4, 5, 6)
```

LIST

list is an ordered, mutable collection of items. You can store elements of any data type—numbers, strings, other lists—and change them after creation. Lists are defined using square brackets and support dynamic resizing.

EXAMPLE

```
   # Empty list
fruits = []
# List with mixed types
data = [1, "apple", 3.14, True]
# List from a range
numbers = list(range(5))  # [0, 1, 2, 3, 4]
```

INTRODUCTION TO COMPUTATIONAL THINKING AND DATA SCIENCE

Introduction and Optimization Problems

Overview of computational thinking and problem-solving techniques.

Introduction to optimization: finding the best solution under constraints.

Optimization Problems

In-depth discussion on types of optimization (linear, non-linear, etc.).

Real-world applications and solving strategies.

Brute-force solution (in Python-like pseudocode):

EXAMPLE

```
   def max_score(F, ps):
  best_score = float('-inf')
  best_combo = None
for x in product([0,1], repeat=5):
 if sum(x) >= 2:
  total_rollover = 10 * sum(x)
  score = (60 - total_rollover) * F + 10 * sum(xi * psi for xi, psi in zip(x,
ps))
 if score > best_score:
  best_score = score
  best_combo = x
  return best_score, best_combo
```

Stochastic Thinking

Understanding randomness and probability in systems
Probabilistic models for decision-making.

EXAMPLE:

```
  import random
def simulate_coin_toss(n):
  heads = 0
for _ in range(n):
 if random.random() < 0.5:
      heads += 1
    return heads / n
#Simulate tossing a coin 1000 times
 trials = 1000 prob_heads = simulate_coin_toss(trials)
print("Stochastic Thinking Example")
print(f"Simulated probability of heads after {trials} tosses:
       {prob_heads:.2f}")
```

Monte Carlo Simulation

Technique using repeated random sampling to solve problems.
Useful in risk analysis and estimating complex systems.

EXAMPLE

```
import random
def monte_carlo_pi(num_samples): inside_circle = 0
for _ in range(num_samples):
    X = random.uniform(0, 1)
    y = random.uniform(0, 1)
distance = x**2 + y**2
if distance <= 1:
       inside_circle += 1
pi_estimate = 4 * inside_circle / num_samples
return pi_estimate
     #Run the simulation with 10000 samples
samples = 10000 estimated_pi = monte_carlo_pi(samples)
print(f"Estimated value of π using {samples} samples: {estimated_pi}")
```

CONCEPT OF PANDAS:

Pandas is a fast, powerful, and flexible open-source data analysis and manipulation tool built on top of **Python**. It is one of the most important libraries in **data science**.

PROGRAM:

```
import os,sys

import pandas as pd

import re

from pathlib import Path

from PySide6.QtWidgets import QFileDialog, QApplication



import openpyxl

from openpyxl.styles import NamedStyle, PatternFill, Font,  Border, Side

from openpyxl.worksheet.table import Table, TableStyleInfo

from openpyxl import load_workbook

from openpyxl.utils import get_column_letter



class FileHandler:

    """Class to handle file selection and directory management."""

    @staticmethod
```

```
    def get_file_path_and_name():
        """Opens a file dialog and returns the selected file path and name."""
        filepath = QFileDialog.getOpenFileName(None, "Select a DAT file", "",
"DAT Files(*.DAT)")
        print(filepath[0])


        if filepath:
            file_path, file_name = os.path.split(filepath[0])
            return file_path, file_name
        else:
            return None, None


class DataProcessor:
    """Class to handle reading, cleaning, and transforming the data."""
    def __init__(self, file_path, file_name):
        self.file_path = file_path
        self.file_name = file_name
        self.df = None
        os.chdir(file_path)


    def read_data(self, cleaned_DAT, delimiter="\t"):
```

```
        """Reads the DAT file into a DataFrame."""

        self.df = pd.read_csv(cleaned_DAT, sep=delimiter, header=[0, 1],
encoding="iso-8859-1")

        self.df.columns = self.df.columns.map(lambda x: tuple(map(str, x)))

        return self.df



    def clean_data(self, cleaned_DAT):

        # Read the file

        with open(self.file_name, 'r') as file:

            lines = file.readlines()



        # Calculate total number of lines

        total_lines = len(lines)



        # Remove the first, second, fifth, and last line

        lines_to_delete = [0, 1, 4, total_lines-1]

        lines = [line for i, line in enumerate(lines) if i not in
lines_to_delete]



        # Write the modified content back to the file

        with open(cleaned_DAT, 'w') as file:
```

```
            file.writelines(lines)

    def transform_data(self, master_file, output_file,cleaned_DAT):
        master_file = master_file
        output_file = output_file

        df_cleaned = self.read_data(cleaned_DAT)
        df_columns = list(df_cleaned.columns[1:])  ### New Line of Code

        # Rename the first column header
        df_cleaned = df_cleaned.rename(
            columns={'Unnamed: 0_level_0': 'WBS_Elements_Info.'})

        # Split "WBS Element Details" column into separate columns
        split_details = df_cleaned[('WBS_Elements_Info.', 'Object')].str.split()

        """Transforms data by splitting the WBS Element Details and renaming
columns to a multi-index format."""
        level, description, id_no = [], [], []
```

```
for details in split_details:

    Level = details[0]

    Description = " ".join(details[1:-2])

    ID = details[-1]



    if Level not in ["*", "**", "***", "4*", "5*"]:

        Description = Level.strip() + Description.strip()

        Level = " "



    level.append(Level)

    description.append(Description)

    id_no.append(ID)



# Add the new columns to the DataFrame

df_cleaned[('WBS_Elements_Info.', 'Level')] = level

df_cleaned[('WBS_Elements_Info.', 'Description')] = description

df_cleaned[('WBS_Elements_Info.', 'ID_No')] = id_no



# Re-arrange columns to place new columns at the beginning
```

```
        df_cleaned = df_cleaned[[('WBS_Elements_Info.', 'Level'),
('WBS_Elements_Info.', 'Description'), ('WBS_Elements_Info.', 'ID_No')] +

                                df_columns ]  # Modified line of code



        # Updating the Description of WBS elements to full description

        master_df = pd.read_excel(master_file)

        transaction_df = df_cleaned



        master_df['WBS_element'] = master_df['WBS_element'].str.strip()

        transaction_df[('WBS_Elements_Info.', 'ID_No')] = transaction_df[(

            'WBS_Elements_Info.', 'ID_No')].str.strip()



        transaction_df[('WBS_Elements_Info.', 'Description')] =
transaction_df[('WBS_Elements_Info.', 'ID_No')].map(

            master_df.set_index('WBS_element')['Name'])



        transaction_df = transaction_df.dropna(how='all')



        # Convert column headers to strings

        # transaction_df.columns = transaction_df.columns.map(str)

        global summary_wbs_list,transaction_wbs_list
```

```
        summary_wbs_list = []

        transaction_wbs_list = []

        summary_wbs_list, transaction_wbs_list =
self.process_wbs(transaction_df[('WBS_Elements_Info.', 'ID_No')].to_list())

        # print("Summary WBS Elements: ",summary_wbs_list,"\n\n Transaction WBS
Elements : ",transaction_wbs_list)

        transaction_df.to_excel(output_file)

        print(f"Updated {output_file} saved successfully!")



    def process_wbs(self, wbs_list):

        """Classifies WBS elements into summary and transaction WBS
dynamically."""

        summary_wbs = []

        transaction_wbs = []



        for wbs in wbs_list:

            # Define a regular expression pattern to match a WBS element with
child elements

            pattern = re.compile(re.escape(wbs) + r"-\d{2}")



            # Check if any WBS starts with the parent WBS followed by a hyphen
and two digits
```

```
            has_child = any(pattern.fullmatch(item) for item in wbs_list if
item != wbs)

            if has_child:
                summary_wbs.append(wbs)
            else:
                transaction_wbs.append(wbs)

        return summary_wbs, transaction_wbs

    def add_heading(self, output_file):
        excel_filename = output_file
        # Load workbook using openpyxl
        wb = load_workbook(excel_filename)
        ws = wb.active  # Get the active sheet

        # Add custom headers in A1 and B1
        ws["A1"] = "Sl"
        ws["A2"] = "No."
```

```
        # Save the modified Excel file
        wb.save(excel_filename)
        return excel_filename


class ExcelFormatter:
    """Class to handle Excel file formatting."""


    def __init__(self, output_file):
        self.output_file = output_file
        self.workbook = openpyxl.load_workbook(output_file)
        self.worksheet = self.workbook.active
        self.last_col = self.worksheet.max_column
        self.last_row = self.worksheet.max_row


    def apply_freeze_panes(self):
        """Freeze panes for the Excel sheet."""
        self.worksheet.freeze_panes = 'E3'


    def apply_font_style(self):
        """Apply a font style to all cells in the worksheet."""
```

```
        font = Font(name='Bookman Old Style', size=12)
        for row in self.worksheet.iter_rows():
            for cell in row:
                cell.font = font

    def apply_currency_formatting(self):
        """Apply currency formatting to specific columns."""
        currency_style = NamedStyle(name='currency',
                                    font=Font(
                                        name='Bookman Old Style', size=12),
                                    number_format=f'₹ #,##0.00;[Red]₹ -#,##0.00')

        for col in range(4, self.last_col+1):
            col_letter = get_column_letter(col)
            column = self.worksheet[col_letter]
            for cell in column:
                cell.style = currency_style

    def adjust_column_width(self):
        for col in range(1, self.last_col + 1):
```

```
            col_letter = get_column_letter(col)

            max_length = max((len(str(cell.value)) for cell in
self.worksheet[col_letter] if cell.value), default=10)

            self.worksheet.column_dimensions[col_letter].width = max_length + 10



    def create_table(self):

        """Create a table in the worksheet from data range."""

        data_range = f"A1:{self.worksheet.cell(row=self.last_row,
column=self.last_col).coordinate}"

        # data_range = "A1:AM690"



        table = Table(displayName="Table1", ref=data_range)

        self.worksheet.add_table(table)



        table.tableStyleInfo = TableStyleInfo(name="TableStyleMedium9",
showFirstColumn=False,

                                              showLastColumn=False,
showRowStripes=True, showColumnStripes=False)



    def apply_header_format(self):

        """Apply specific formatting to the header cells."""

        last_col_letter = get_column_letter(self.last_col)
```

```
        # lastPos= last_col_letter+str(self.last_row)



        # Define Black Border

        black_border = Border(

            left=Side(style="thin", color="000000"),

            right=Side(style="thin", color="000000"),

            top=Side(style="thin", color="000000"),

            bottom=Side(style="thin", color="000000"),

)




        cell_range_str = f"A1:{self.worksheet.cell(row=self.last_row,
column=self.last_col).coordinate}"

        cell_range = self.worksheet[cell_range_str]



        yellow_fill = PatternFill(start_color='FFFF00',

                                  end_color='FFFF00', fill_type='solid')



        skyBlue_fill = PatternFill(start_color='87CEEB',
```

```
                                      end_color='87CEEB', fill_type='solid')

        white_fill = PatternFill(start_color='FFFFFF',

                                      end_color='FFFFFF', fill_type='solid')

        rowCtr = 0

        # White text

        bold_font = Font(name='Bookman Old Style', size=12, color="000000", bold
= True)

        for row in cell_range:

            for cell in row:

                if rowCtr in [0, 1]:

                    cell.fill = yellow_fill

                    cell.border = black_border

                    cell.font = bold_font

                else:

                    if rowCtr  % 2 == 0:

                        cell.fill = skyBlue_fill

                        cell.border = black_border
```

```
                else:
                    cell.fill = white_fill
                    cell.border = black_border
                    # cell.font = white_font
            rowCtr += 1

    def highlight_rows_by_value(self, values_list):
        """
        For each cell in column 'D', if the cell's value is in values_list,
        highlight the entire row with a yellow fill.

        Parameters:
        self.worksheet          : openpyxl worksheet object
        values_list : list of values to check against
        """
        light_green_fill = PatternFill(start_color="90EE90",
                                       end_color="90EE90", fill_type="solid")

        # self.worksheet['D'] returns all cells in column D
        for cell in self.worksheet['D']:
```

```
        if cell.value in values_list:

            # Iterate through all cells in the current row and apply the
yellow fill

            for row_cell in self.worksheet[cell.row]:

                row_cell.fill = light_green_fill



    def save(self):

        """Save the final formatted workbook."""

        self.workbook.save(self.output_file)



def main():

    app = QApplication(sys.argv)

    cleaned_DAT = 'temp.dat'

    master_file = 'WBS_NAMES.XLSX'



    # Step 1: File selection

    file_path, file_name = FileHandler.get_file_path_and_name()

    os.chdir(file_path)



    # Change the file extension to .xlsx

    output_file = Path(file_name).with_suffix(".XLSX")
```

```python
# Step 2: Data processing

data_processor = DataProcessor(file_path, file_name)

data_processor.clean_data(cleaned_DAT)

data_processor.transform_data(master_file, output_file,cleaned_DAT)

data_processor.add_heading(output_file)


# Step 3: Excel formatting

excel_formatter = ExcelFormatter(output_file)

excel_formatter.apply_freeze_panes()

excel_formatter.apply_font_style()

excel_formatter.apply_currency_formatting()

excel_formatter.adjust_column_width()

# excel_formatter.create_table()

excel_formatter.apply_header_format()


# print(summary_wbs_list)

excel_formatter.highlight_rows_by_value(summary_wbs_list)

excel_formatter.save()
```

```python
    print(
        f"Excel file '{output_file}' has been created with the specified formatting.")
    # sys.exit(app.exec())


if __name__ == "__main__":
    main()
```

Program explanation

This project, BudgetReport.py, is a Python application designed to process and format budget data. It reads a raw .DAT file, cleans and transforms the data, and then outputs a meticulously formatted Excel spreadsheet (.xlsx). The script leverages several libraries, including pandas for data manipulation, openpyxl for Excel formatting, and PySide6 for the file dialog.

Project Structure

The project is organized into several classes, each responsible for a specific aspect of the data processing and formatting pipeline:

- **FileHandler**: Manages file selection and directory operations.
- **DataProcessor**: Handles reading, cleaning, and transforming the budget data.
- **ExcelFormatter**: Applies extensive formatting to the generated Excel file.

Class-wise Explanation

FileHandler Class

This class is responsible for interacting with the user to select the input .DAT file and managing file paths.

- **get_file_path_and_name()**:

    - Opens a file dialog, allowing the user to select a .DAT file.
    - Prints the selected file path to the console.
    - Returns the directory path and the name of the selected file.
    - If no file is selected, it returns None for both the path and the name.

DataProcessor Class

This class handles the core logic of reading, cleaning, and transforming the raw budget data.

- **__init__(self, file_path, file_name)**:
    - Initializes the DataProcessor with the provided file_path and file_name.
    - Sets the current working directory to file_path.
    - Initializes self.df to None, which will hold the DataFrame.
- **read_data(self, cleaned_DAT, delimiter="\t")**:
    - Reads the cleaned_DAT file into a pandas DataFrame.
    - Assumes a tab delimiter and reads the header from the first two rows.
    - Converts column names to tuples of strings for multi-index compatibility.
    - Returns the populated DataFrame.
- **clean_data(self, cleaned_DAT)**:
    - Reads the content of the original .DAT file.
    - Removes specific lines (first, second, fifth, and last) from the file, as these are typically header/footer information not needed in the processed data.
    - Writes the modified content to a new file specified by cleaned_DAT.
- **transform_data(self, master_file, output_file, cleaned_DAT)**:
    - This is the central data transformation method.
    - Reads the cleaned data from cleaned_DAT into df_cleaned.
    - Renames the first column ('Unnamed: 0_level_0', 'Object') to ('WBS_Elements_Info.', 'Object').
    - Splits the ('WBS_Elements_Info.', 'Object') column into Level, Description, and ID_No based on space delimiters and specific patterns for WBS elements.

    - Reorganizes the DataFrame columns to place the newly created Level, Description, and ID_No at the beginning.
    - Reads a master_file (an Excel file, e.g., WBS_NAMES.XLSX) containing WBS element names.
    - Maps the ID_No from the transaction_df to the Name from the master_df to update the Description column with full descriptions.
    - Drops rows that are entirely empty after the mapping.
    - Calls process_wbs to classify WBS elements into summary_wbs_list and transaction_wbs_list based on whether they have child elements. These lists are made global for access by other functions.
    - Saves the transformed DataFrame to an Excel file specified by output_file.
    - Prints a success message upon saving.
- **process_wbs(self, wbs_list)**:
    - Takes a list of WBS elements as input.
    - Iterates through each WBS element.
    - Uses a regular expression to determine if a WBS element has child elements (e.g., WBS-01).
    - Classifies WBS elements into summary_wbs (those with children) and transaction_wbs (those without children).
    - Returns two lists: summary_wbs and transaction_wbs.
- **add_heading(self, output_file)**:
    - Loads the Excel workbook specified by output_file.
    - Adds "Sl" to cell A1 and "No." to cell A2.
    - Saves the modified Excel file.

ExcelFormatter Class

This class is responsible for applying various formatting styles to the generated Excel file, enhancing its readability and presentation.

- **__init__(self, output_file)**:
    - Initializes the ExcelFormatter with the output_file path.
    - Loads the Excel workbook and selects the active worksheet.
    - Determines last_col and last_row for dynamic range calculations.
- **apply_freeze_panes(self)**:

    - Freezes the panes at cell 'E3', which means rows 1 and 2 and columns A, B, C, D will remain visible when scrolling.
- **apply_font_style(self)**:
    - Applies 'Bookman Old Style' font with size 12 to all cells in the worksheet.
- **apply_currency_formatting(self)**:
    - Defines a custom currency style (₹ #,##0.00;[Red]₹ -#,##0.00) to display monetary values in Indian Rupees (₹), with negative values shown in red.
    - Applies this currency style to all cells from column 4 (D) to the last column.
- **adjust_column_width(self)**:
    - Iterates through each column.
    - Calculates the maximum length of content within each column.
    - Adjusts the column width to accommodate the content plus an additional 10 units for padding.
- **create_table(self)**:
    - Creates an Excel table named "Table1" from the data range starting at A1 to the last cell containing data.
    - Applies a TableStyleMedium9 style, showing row stripes but not first/last columns or column stripes.
    - *Note: This method is commented out in main(), so it's not currently active.*
- **apply_header_format(self)**:
    - Defines a thin black border style.
    - Defines yellow_fill, skyBlue_fill, and white_fill patterns.
    - Defines a bold font style for headers.
    - Iterates through all cells in the data range.
    - Applies yellow fill, black borders, and bold font to the first two rows (headers).
    - Applies alternating skyBlue_fill and white_fill with black borders to subsequent rows, creating a striped effect.
- **highlight_rows_by_value(self, values_list)**:
    - Defines a light_green_fill pattern.

    - Iterates through each cell in column 'D' (WBS_Elements_Info.ID_No).
    - If a cell's value is found in values_list (which will be summary_wbs_list), the entire row containing that cell is highlighted with light_green_fill. This visually distinguishes summary WBS elements.
- **save(self)**:
    - Saves the modified workbook to the output_file.

main() Function

This function orchestrates the entire workflow of the application.

1. **Initialize QApplication**: Creates a QApplication instance, which is necessary for QFileDialog.
2. **Define Constants**: Sets cleaned_DAT to 'temp.dat' (an intermediate file) and master_file to 'WBS_NAMES.XLSX'.
3. **File Selection**: Calls FileHandler.get_file_path_and_name() to allow the user to select the input .DAT file.
4. **Set Working Directory**: Changes the current directory to the selected file_path.
5. **Determine Output File Name**: Derives the output_file name by changing the extension of the input file to .XLSX.
6. **Data Processing**:
    a. Creates an instance of DataProcessor.
    b. Calls clean_data() to clean the raw DAT file.
    c. Calls transform_data() to process and transform the data, including splitting WBS elements, mapping descriptions, and classifying WBS types.
    d. Calls add_heading() to add custom headers.
7. **Excel Formatting**:
    a. Creates an instance of ExcelFormatter.
    b. Applies various formatting functions: apply_freeze_panes(), apply_font_style(), apply_currency_formatting(), adjust_column_width(), apply_header_format().

   c. Calls highlight_rows_by_value() to highlight summary WBS rows based on the summary_wbs_list generated during data transformation.
   d. Calls save() to save the final formatted Excel file.
8. **Confirmation**: Prints a message indicating that the Excel file has been created.

Global Variables

- **summary_wbs_list**: A global list populated by DataProcessor.process_wbs() containing WBS elements identified as "summary" WBS (those having child elements). This list is used by ExcelFormatter.highlight_rows_by_value() to visually mark these rows in the Excel output.
- **transaction_wbs_list**: A global list also populated by DataProcessor.process_wbs() containing WBS elements identified as "transaction" WBS (those without child elements). This list is currently not used for formatting.

Dependencies

The script relies on the following Python libraries:

- os: For interacting with the operating system, like changing directories.
- sys: For system-specific parameters and functions, used for QApplication.
- pandas: A powerful data manipulation and analysis library.
- re: Regular expression operations, used for WBS classification.
- pathlib.Path: For object-oriented filesystem paths.
- PySide6.QtWidgets.QFileDialog, QApplication: For creating GUI elements, specifically the file selection dialog.
- openpyxl: A library for reading, writing, and modifying Excel .xlsx files. This includes various modules for styles, tables, and utility functions.

How to Run

1. Ensure all required Python libraries are installed (pip install pandas openpyxl PySide6).
2. Have a WBS_NAMES.XLSX file in the same directory as the script, containing WBS_element and Name columns.

3. Execute the script (python BudgetReport.py).
4. A file dialog will appear, prompting the user to select the input .DAT file.
5. The script will process the data and generate a new .xlsx file with the same name as the input .DAT file (but with the .xlsx extension) in the same directory. A temporary temp.dat file will also be created and used during the process.

CONCLUSION:

In conclusion, the SAP PROJECT SYSTEM REPORTS USING PYTHON DATASCIENCE TOOLS internship has been a highly enriching and valuable experience for the past 2 weeks . Over the course of this internship, I had the opportunity to gain practical exposure to the SAP ERP system, specifically focusing on the web development functionalities. The internship has provided me with a unique insight into the world of enterprise resource planning and how it plays a crucial role in optimizing business processes. During my tenure as an intern, I was able to actively participate in various web development projects, which involved working with different modules like Financial Accounting, Inventory Management, and Human Capital Management. Through hands-on tasks, I have developed a deeper understanding of how data is managed, processed, and utilized to drive informed decision-making within an organization. One of the key takeaways from this internship has been the importance of accurate and timely information in the decision-making process. SAP's web functionalities have proven to be an indispensable tool for businesses to monitor performance, analyze trends, and identify areas for improvement. Moreover, I have gained proficiency in generating insightful reports and dashboards, providing valuable insights to key stakeholders. Additionally, the collaborative work environment and the support of my mentor and colleagues have been instrumental in enhancing my learning experience. The constant feedback and encouragement have allowed me to grow both professionally and personally. As the internship comes to an end, I am confident that the skills and knowledge gained during this two weeks journey will serve as a strong foundation for my future career endeavors. The exposure to SAP's numpy ,pandas has sparked my interest in enterprise systems, and We are eager to further explore this field. I extend my heartfelt gratitude to the entire team at Enterprise Resource Planning, NLCIL for providing us with this excellent opportunity. This internship has undoubtedly been a stepping stone in our professional growth, and I look forward to apply the lessons learned here to make meaningful contributions to any future organization we join

REFERENCES:

https://ocw.mit.edu/courses/6-0001-introduction-to-computer-science-and programming-in-python-fall-2016/

https://www.edx.org/learn/data-science/harvard-university-introduction-to-data-science-with-python

https://www.youtube.com/watch?v=hNCd_BM4T3M&t=6100s

https://www.youtube.com/watch?v=vtgDGrUiUKk&t=1703s

FEEDBACK:

the data science using python for data analysis internship was an exceptional learning experience. I gained valuable insights into the SAP ERP system and its impact on business processes. The supportive mentor made the journey even more rewarding. I am thankful for this enriching experience and look forward to leveraging this knowledge in my professional path. Thank you for an outstanding internship opportunity! NLC INDIA LIMITED("NAVRATNA" – A GOVERNMENT OF INDIA ENTERPRISE")NEYVELI – 607 803, TAMIL NADU